Лабораторная работа № 5
Управление ресурсами в Kubernetes

Смирнов Александр, группа J3310
Вариант № 11: DevOps
9 ноября 2026 г.

1. Цель и устройство стенда

Цель работы - запустить расчёт пар навыков из ДЗ 3 в Kubernetes, подобрать ограничение памяти по измерениям и проверить планирование при недостатке ресурсов, приоритеты и квоты.

Компонент	Конфигурация
Среда	Windows, Docker Desktop 29.1.2, WSL2
Кластер	kind 0.33.0, Kubernetes и kubectl 1.37.0; control-plane и один worker
Расчёт	Python 3.12.12, boto3 1.43.114, образ lab5-pairs:1
Хранилище	RustFS 1.0.0, Service rustfs:9000, PVC 1 ГиБ
Данные	6 215 вакансий DevOps, NDJSON 1 880 228 байт

Worker имеет метку lab=compute. На компьютере Docker видит 32 CPU; через system-reserved оставлены 2 CPU для планирования Pod на worker. Это управляемое ограничение стенда, а не измерение загрузки процессора. Control-plane не принимает расчётные Pod. Все опыты выполняются в namespace lab5.

Порядок выполнения

Сначала Job считает пары из файла внутри образа. Затем тот же файл загружается в RustFS в кластере. Indexed Job запускает четыре Mapper, каждый читает свой диапазон S3 Range GET и сохраняет частичный результат. После завершения всех частей отдельный Job суммирует частоты и публикует pairs.json.

```
NDJSON -> Job: измерение памяти и подбор limit
NDJSON -> RustFS -> Indexed Job (4 части) -> Reducer -> S3
PriorityClass / requests -> планирование; ResourceQuota -> допуск
```

Расчёт использует ту же нормализацию, что и ДЗ 3: уникальные навыки вакансии сортируются, пары строятся без повторений и перестановок. Порядок итоговых строк: частота по убыванию, затем названия навыков. Файлы данных и эталона включены в проект.

```
powershell -ExecutionPolicy Bypass -File .\bootstrap.ps1
python verify_results.py
python -m unittest discover -s tests -v
```

2. Остановка по памяти и выбор предела

Job pairs-oom получает requests.cpu = limits.cpu = 250m и requests.memory = limits.memory = 40Mi. Автоматические повторы отключены (backoffLimit: 0, restartPolicy: Never), чтобы сохранить причину завершения первого процесса.

```
kubectl --context kind-lab5 logs job/pairs-oom -n lab5
{"event": "started", "mode": "local"}

Состояние контейнера из logs/pairs-oom-pods.json:
reason: OOMKilled
exitCode: 137
startedAt: 2026-11-09T13:00:19Z
finishedAt: 2026-11-09T13:00:20Z
```

Сообщение started появляется до чтения и обработки файла. Таким образом, контейнер был остановлен уже после запуска программы. Код завершения 137 сам по себе недостаточен для вывода об OOM; здесь Kubernetes явно указал OOMKilled.

Для измерения выполнены три успешных запуска с верхним пределом 256 МиБ и заявкой 64 МиБ. RSS получен через getrusage; пик памяти контейнера - из memory.peak cgroup v2 перед выводом итоговой строки. Время ниже относится к вычислению, без ожидания Pod и импорта модулей.

Запуск	Время, с	Пик RSS, МиБ	Пик cgroup, МиБ
1	0.447	53.21	42.53
2	0.405	53.23	42.48
3	0.486	53.71	42.48

Наибольший измеренный пик cgroup равен 42.53 МиБ. Добавляю 25% запаса и округляю вверх до кратного 16 МиБ: ceil(42.53 × 1,25 / 16) × 16 = 64 МиБ. Контрольный Job pairs-sized с этим пределом завершился успешно; пик cgroup составил 42.51 МиБ. Все частоты совпали с эталоном.

RSS процесса и память cgroup учитывают разделяемые страницы по-разному, поэтому их значения не обязаны совпадать. Для лимита контейнера использован показатель cgroup. Измерение относится к этому образу и датасету; увеличение входа требует повторного профилирования.

В Kubernetes requests участвуют в выборе узла, а limits ограничивают потребление во время работы. Превышение CPU обычно приводит к throttling; превышение лимита памяти может завершиться OOMKilled. [1]
В командах отчёта kubectl означает закреплённый tools/kubectl.exe версии 1.37.0.

3. RustFS и параллельный расчёт

RustFS развёрнут внутри Kubernetes как Deployment с одним экземпляром и постоянным томом PVC. Service предоставляет S3 API по адресу http://rustfs:9000. В бакет hh-lake загружен исходный файл ДЗ 3; его SHA256 проверен. Это перенос сервиса и нужных расчёту данных, а не копирование всех объектов старого бакета.

Indexed Job содержит completions: 4 и parallelism: 4. Каждый Mapper имеет CPU 250m и предел памяти 128Mi; индекс берётся из JOB_COMPLETION_INDEX. Запись принадлежит диапазону, содержащему её первый байт. Конец последней строки дочитывается адаптивно. Повтор Mapper заменяет объект своего индекса, поэтому Reducer учитывает каждую часть один раз.

Индекс	Диапазон байт включительно	Вакансий	Уникальных пар
0	0 - 470056	1554	9766
1	470057 - 940113	1602	10034
2	940114 - 1410170	1548	11474
3	1410171 - 1880227	1511	10814

Сумма: 6 215 вакансий, 173 204 вхождения пар; после объединения 25 462 уникальные пары. Итог скачан из S3 и полностью сопоставлен с reference/lab3_pairs.json. Совпали все частоты и приведённый ниже топ-10.

№	Пара навыков	ДЗ 3	Kubernetes
1	docker + linux	1583	1583
2	docker + kubernetes	1549	1549
3	kubernetes + linux	1444	1444
4	ansible + linux	1069	1069
5	devops + linux	1032	1032
6	ansible + docker	990	990
7	devops + docker	990	990
8	devops + kubernetes	966	966
9	ansible + kubernetes	940	940
10	linux + python	922	922

SHA256 канонического массива counts (UTF-8, JSON без пробелов):

`6c6badc5fe61c28abe84df73a97880bcdb9949b2cda47e02db1e9f5ed11bb838`

В S3 сохранены input/vacancies.ndjson, четыре part-N.json и runs/parallel/pairs.json. Полный результат также находится в results/pairs.json. Пять регрессионных тестов проверяют границы строк, длинную запись, нормализацию, ошибочный вход и совпадение с ДЗ 3. Локальный PVC сохраняется при перезапуске Pod, но не гарантирует сохранность при удалении kind-кластера.

4. Ожидание CPU, вытеснение и квота

Для опытов с планированием режим hold удерживает заявку на ресурсы, не нагружая CPU вычислениями. Это позволяет наблюдать именно решения планировщика. Подсчёт пар к этому моменту уже завершён.

Недостаток CPU

Pod low с приоритетом 10 занимает заявку 1200m. RustFS запрашивает ещё 250m. Второму Pod waiting с заявкой 1200m не хватает ресурсов worker; он остаётся Pending. Свободный CPU по фактической загрузке не отменяет уже выданные заявки.

```
kubectl --context kind-lab5 describe pod waiting -n lab5
FailedScheduling: 0/2 nodes are available:
1 Insufficient cpu, 1 node(s) had untolerated taint(s).
```

Вытеснение по приоритету

Pod high запрашивает те же 1200m, но использует PriorityClass со значением 100 и preemptionPolicy: PreemptLowerPriority. Планировщик вытесняет low и запускает high. waiting с приоритетом 10 остаётся Pending. [2]

```
Событие из logs/preemption-events.txt:
reason: Preempted
object: low
Preempted by pod 9233e471-086b-4ff9-a358-57d948a3affe on node lab5-worker

Фрагмент kubectl get pods после вытеснения:
high       1/1   Running
waiting    0/1   Pending
```

Квота пространства имён

После освобождения тестовых Pod создана ResourceQuota: requests.cpu = 750m, requests.memory = 512Mi, limits.memory = 2Gi, pods = 3. RustFS занимает 250m; Pod quota-ok добавляет 400m. Следующий Pod с заявкой 400m отклонён на этапе допуска, ещё до планирования.

```
Error from server (Forbidden): exceeded quota: compute-budget
requested: requests.cpu=400m
used: requests.cpu=650m
limited: requests.cpu=750m
```

Это три разных механизма: Pending означает, что существующему Pod не найден узел; preemption освобождает заявки вытеснением менее приоритетного Pod; ResourceQuota запрещает создание объекта сверх бюджета namespace. Квота не добавляет CPU и не заменяет проверку доступности узла. [3]

Полные команды и ответы сохранены в logs/cpu-waiting.txt, logs/after-preemption.txt, logs/preemption-events.txt, logs/apply-quota-rejected.txt и logs/quota-status.txt. Приведённые выше строки сокращены только для чтения.

5. Соответствие YARN, Mesos и Kubernetes

Сравнение основано на лекции 5 и документации систем. Соответствия отражают роли компонентов, а не полную взаимозаменяемость. [4-6]

Роль	YARN	Mesos	Kubernetes
Распределение ресурсов	ResourceManager / Scheduler	Master / allocator	kube-scheduler выбирает узел для Pod
Агент узла	NodeManager	Agent (ранее slave)	kubelet и container runtime
Координация расчёта	ApplicationMaster на приложение	Framework scheduler + executors	Job controller и логика приложения
Единица работы	Container с ресурсами; не обязательно Docker	Task через executor	Pod с одним или несколькими контейнерами
Выделение	Запросы контейнеров от AM	Resource offers; framework принимает или отклоняет	requests и constraints Pod; limits при исполнении
Бюджеты и приоритеты	Очереди Capacity/Fair, preemption	Роли, квоты, политика allocator	ResourceQuota и PriorityClass

Ответы на самопроверку из лекции

Зачем нужен менеджер ресурсов? Он учитывает мощности узлов и распределяет их между приложениями. При раздельных кластерах свободные машины MapReduce нельзя сразу отдать перегруженному Spark; общий пул уменьшает простой и необходимость копировать данные.

Чем отличаются подходы? В YARN центральный Scheduler распределяет контейнеры по запросам приложений; AM определяет задачи приложения. В Mesos allocator предлагает ресурсы фреймворкам, а их планировщики выбирают задачи и размещение в пределах предложения. В Kubernetes обычный kube-scheduler централизованно назначает узел Pod, но не управляет алгоритмом расчёта.

Три сущности YARN. ResourceManager распределяет ресурсы кластера; NodeManager запускает и отслеживает контейнеры узла; ApplicationMaster запрашивает ресурсы и координирует задачи одного приложения. Job controller Kubernetes похож только частью обязанностей: он поддерживает число запусков, а объединение наших результатов выполняет отдельная программа.

Цикл resource offer. Agent сообщает ресурсы master; allocator предлагает их framework scheduler; тот принимает предложение с описанием задач либо отклоняет. Master передаёт задачи agent, а executor исполняет их. После освобождения ресурсов цикл повторяется.

Отказоустойчивость Mesos. Несколько master участвуют в выборе активного через ZooKeeper. При потере его сессии выбирается новый лидер. ZooKeeper координирует выбор, но не исполняет задачи и не заменяет сохранение состояния приложения. Kubernetes хранит состояние в etcd и использует свои механизмы лидерства компонентов.

6. Обсуждение и выводы

Почему двухуровневое планирование удобно при появлении новых фреймворков? Каждый framework scheduler реализует собственные требования к задачам и локальности без переноса всей логики в общий планировщик. Master сохраняет контроль распределения ресурсов между фреймворками. Цена такой гибкости - дополнительные переговоры и менее полный глобальный обзор решений.

Что произойдёт с пятисекундным запросом при большом задании FIFO? Если кластер занят, запрос ждёт освобождения ресурсов. Fair Scheduler распределяет доли между приложениями, а Capacity Scheduler позволяет выделить очередь интерактивным задачам. Для быстрого освобождения уже занятых ресурсов нужны настроенные гарантии и preemption; смена названия планировщика сама по себе не обеспечивает немедленный запуск. [7]

Простаивают ли ресурсы после отказа framework A от предложения? Нет. Allocator может предложить их другому фреймворку. Фильтр отказа может временно исключить повтор того же предложения для A, но не резервирует ресурсы за ним. [5]

Пример объединения двух кластеров. Пусть MapReduce и Spark используют по четыре одинаковых узла. После объединения восемь узлов работают под YARN с общей доступностью данных. Очередям задаются гарантии по 50% и возможность заимствовать свободную долю. Когда MapReduce простаивает, Spark может использовать больше половины ресурсов; при возврате MapReduce планировщик восстанавливает доли согласно политике preemption. Это пример организации, а не эксперимент, выполненный на данном стенде.

Результат работы

Kubernetes остановил расчёт при недостаточном лимите памяти и успешно выполнил его с пределом 64 МиБ, выбранным по измеренному пику. Перенос RustFS и разбиение файла на четыре диапазона сохранили все частоты ДЗ 3. Эксперименты отдельно подтвердили ожидание CPU, вытеснение менее приоритетного Pod и отказ при превышении квоты.

Стенд локальный: два узла kind работают на одном компьютере, а расчётные Pod размещаются на одном worker. Это демонстрация параллельного исполнения и управления ресурсами, а не проверка отказоустойчивости нескольких физических машин. Ускорение от четырёх Pod не заявляется: на небольшом входе запуск контейнеров и обращения к S3 могут стоить больше самого подсчёта.

Источники

[1] Kubernetes. Resource Management for Pods and Containers. kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[2] Kubernetes. Pod Priority and Preemption. kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/

[3] Kubernetes. Resource Quotas. kubernetes.io/docs/concepts/policy/resource-quotas/

[4] Apache Hadoop. YARN. hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html

[5] Apache Mesos. Architecture. mesos.apache.org/documentation/latest/architecture/

[6] К. Хицко. Лекция 5 «Менеджеры ресурсов YARN и Mesos». Слайды 10, 17, 22, 24, 28.

[7] Apache Hadoop. Fair Scheduler и Capacity Scheduler. hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/FairScheduler.html